Multimedia Signal Processing - 2nd module

Prof. Marco Tagliasacchi

February, 16 2010

1 Question - 10 pts

A wireless sensor needs to communicate to the base station the state of the processes running on its CPU. Each process can be either in run (R), wait (W) or sleep (S) state. Figure 1 illustrate the transition probabilities $p(s_t|s_{t-1})$. The sampling frequency is 10 Hz. The marginal probabilities can be determined to be

$$p(s_t = R) = 0.2, \quad p(s_t = W) = 0.2, \quad p(s_t = S) = 0.6$$

Figure 1: Each node represent a state, each edge a state transition with the associated probability.

- Compute the entropy of the source X (in bits/symbol) that emits symbols from the alphabet $\{R, W, S\}$, assuming that $p(s_t|s_{t-1}) = p(s_t)$ (discrete memoryless source).
- Simulate the encoding and decoding of the sequence $SSWR$ using Elias coding in three cases:
  - Assuming that $p(s_t|s_{t-1}) = p(s_t)$ (discrete memoryless source).
  - Assuming the knowledge of the conditional probabilities as in Figure 1.
  - Assuming that the source statistics are unknown and they are determined at encoding time as symbols are observed.

  Compare the lengths of the associated codeword in the three cases.
- When the sensor is in running state, it communicates its reading, quantizing it with $R = 4$ bits/symbol. Compute the overall data rate (to communicate both the sequence of states and the readings) when source memory is not exploited and there are $m = 10$ processes.

Solution

- The entropy of the source is

$$H(X) = -0.2 \log_2 0.2 - 0.2 \log_2 0.2 - 0.6 \log_2 0.6 = 1.371 \tag{1}$$

- Assuming a DMS, the unit interval is partitioned in three segments $[0, 0.2)$, $[0.2, 0.4)$, $[0.4, 1)$.

symbol	interval	length	p
S	$[0.4, 1)$	0.6	$p(S) = 0.6$
S	$[0.64, 1)$	0.36	$p(S) = 0.6$
W	$[0.712, 0.784)$	0.072	$p(W) = 0.2$
R	$[0.712, 0.7264)$	0.0144	$p(R) = 0.2$

  Consider a point $x = 0.72$ included in the $[0.712, 0.7264)$ interval. We need to retain the first $l = -\lceil \log_2 0.0144 \rceil + 1 = 8$ bits of the binary expansion.

The binary expansion of 0.72 can be found as follows:

0.7200	×2	1.4400	1	1/2
0.4400	×2	0.8800	0	0
0.8800	×2	1.7600	1	1/8
0.7600	×2	1.5200	1	1/16
0.5200	×2	1.0400	1	1/32
0.0400	×2	0.0800	0	0
0.0800	×2	0.1600	0	0
0.1600	×2	0.3200	0	0

The bistream is 10111000. Decoding proceed as follows.

- Find the number corresponding to the bitstream. That is, $1/2 + 1/8 + 1/16 + 1/32 = 0.7188$.
- Decode symbols (the length of the sequence needs to be known)

I_R	I_W	I_S	decoded
$[0, 0.4)$	$[0.2, 0.4)$	$[\mathbf{0.4}, \mathbf{1})$	S
$[0.4, 0.52)$	$[0.52, 0.64)$	$[\mathbf{0.64}, \mathbf{1})$	S
$[0.64, 0.712)$	$[\mathbf{0.712}, \mathbf{0.784})$	$[0.784, 1)$	W
$[\mathbf{0.712}, \mathbf{0.7264})$	$[0.7264, 0.7408)$	$[0.7408, 0.784)$	R

- Assuming knowledge of the conditional probabilities. The first symbol is encoded based on the marginals.

symbol	interval	length	p
S	$[0.4, 1)$	0.6	$p(S) = 0.6$
S	$[0.46, 1)$	0.54	$p(S\|S) = 0.9$
W	$[0.5140, 0.5680)$	0.054	$p(W\|S) = 0.1$
R	$[0.5140, 0.5194)$	0.0054	$p(R\|W) = 0.1$

Consider a point $x = 0.515$ included in the $[0.5140, 0.5194)$ interval. We need to retain the first $l = -\lceil \log_2 0.0054 \rceil + 1 = 9$ bits of the binary expansion.
Encoding and decoding proceeds according to the same scheme shown for the DMS case.

- We initialize three histograms, to keep track of the statistics. That is,

$$n(R|R) = k, n(W|R) = k, n(S|R) = k \tag{2}$$

$$n(R|W) = k, n(W|W) = k, n(S|W) = k \tag{3}$$

$$n(R|S) = k, n(W|S) = k, n(S|S) = k \tag{4}$$

For example, we set $k = 1$.

sym.	interval	length	$R\|R$	$W\|R$	$S\|R$	$R\|W$	$W\|W$	$S\|W$	$R\|S$	$W\|S$	$S\|S$	p
S	$[0.6667, 1)$	0.3333	1	1	1	1	1	1	1	1	1	$p(S) = 1/3$
S	$[0.8889, 1)$	0.1111	1	1	1	1	1	1	1	1	2	$p(S\|S) = 1/3$
W	$[0.9168, 0.9445)$	0.0277	1	1	1	1	1	1	1	2	2	$p(W\|S) = 1/4$
R	$[0.9168, 0.9260)$	0.0092	1	1	1	1	1	1	1	2	2	$p(R\|W) = 1/3$

Note that encoding of a symbols exploits the statistics collected up to the previous symbol.
Consider a point $x = 0.917$ included in the $[0.9168, 0.9260)$ interval. We need to retain the first $l = -\lceil \log_2 0.0092 \rceil + 1 = 8$ bits of the binary expansion.
Encoding and decoding proceeds according to the same scheme shown for the DMS case.

- The overall bit budget includes
  - The cost of sending the state, i.e. $R_{state} = 1.371$ bits / symbol. At 10Hz, we obtain 13.71 bits / second.
  - The cost of the reading, when in running state. That is, 4 bits / symbol with probability $p = 0.2$, i.e. $4 \cdot 0.2 = 0.8$ bits / symbol. At 10Hz, we obtain $R_{data} = 8$ bits / symbol.

Since there are $m = 10$ processes, we spend $10 \cdot (13.71 + 8) = 217.1$ bits / second.

2 Question - 5 pts

Illustrate the design of the Lloyd-Max quantizer. Compare the rate-distortion performance of the Lloyd-Max quantizer with other possible quantizer designs.

Solution

See slide set `MMSP_1.2_Lossy coding`.

3 Questions - 5 pts (each answer can be either TRUE or FALSE)

1. Let X denote the source of Exercise 1. Assume X be a discrete memoryless source
   - [T] [F] $H(X) = 0.4127$ bits/symbol
     FALSE.
   - [T] [F] $i(W) \geq i(S)$, where $i(\cdot)$ is the information content of a symbol
     TRUE. $i(a) = -\log_2 p(a)$
   - [T] [F] The average code length of Shannon coding groups of $N = 3$ symbols is less than 1.7043 bits/symbol
     TRUE. $H(X) + 1/3 = 1.7043$
   - [T] [F] $l(W) \leq l(S)$, where $l(\cdot)$ is the codeword length adopted by Huffman coding
     FALSE. The length is inversely proportional to the probability of a symbols. Since $p(W) > p(S)$, $l(S) \leq l(W)$ (but is might be that $l(S) = l(W)$)
2. Consider a speech coding system
   - [T] [F] The long term predictor is useful when coding unvoiced speech
     FALSE. The long term predictor exploits periodic correlation in voiced speech.
   - [T] [F] Direct quantization of prediction coefficients might lead to instability at the decoder
     TRUE. Alternative quantization schemes (e.g. based on reflection coefficients) need to be used to ensure stability.
   - [T] [F] The number of poles of the whitening filter $A(z)$ depends on the number of formants below 4 kHz
     FALSE. $A(z)$ is an all-zero filter.
3. Consider a block-based image coding system
   - [T] [F] With threshold quantization, each block is allocated the same bit budget
     FALSE. The actual number of bits varies depending on the content of the block.
   - [T] [F] Increasing the block size, the transform coding gain increases
     TRUE. In principle, higher coding efficiency can be achieved exploiting long-range inter-pixel correlations (at the cost of increased complexity)
   - [T] [F] DPCM can be useful to exploit inter-block correlations.
     TRUE. Inter-block correlations exist for DC coefficient. JPEG uses DPCM to encode the sequence of DC coefficients across blocks.

4 Question - 12 pts

Load the image by means of the Matlab command `imread('rice.png')`. Let X denote a source that emits the pixel values of the image from the alphabet $\{0, 1, \ldots, 255\}$.

- Compute the empirical entropy of the source X, assuming that symbols are statistically independent
- Compute the empirical entropy of the source X, conditioned on the knowledge of the value of the pixel located either above or at the left of the current pixel
- Implement a DPCM codec that exploits the best between the two predictors and plot the rate-distortion curve in a PSNR (dB) -rate (bpp) chart.

Solution

```
clear all
close all
clc;

I = double(imread('rice.png'));
[M N] = size(I);

%entropy of the memoryless source
X = I(:);
p = hist(X,0:255);
p = p./sum(p);
p = p(p~=0);
H = -sum(p.*log2(p))

%conditional entropies (up or left pixels)
P_up = zeros(256,256);
P_left = zeros(256,256);

%UP: column-wise "unroll" of the input image
Xup = I(:);

%LEFT: row-wise
Xleft = I';
Xleft = Xleft(:);

%compute joint entropies
for ip = 2:M*N
    P_up(Xup(ip),Xup(ip-1)) = P_up(Xup(ip),Xup(ip-1))+1;
    P_left(Xleft(ip),Xleft(ip-1)) = P_left(Xleft(ip),Xleft(ip-1))+1;
end

P_up = P_up(:)./(M*N);
P_up = P_up(P_up~=0);

P_left = P_left(:)./(M*N);
P_left = P_left(P_left~=0);

H_joint_up = -sum(P_up.*log2(P_up));
H_joint_left = -sum(P_left.*log2(P_left));


%compute conditional entropies
H_cond_up = H_joint_up - H
H_cond_left = H_joint_left-H


%From the inspection of H_cond_up and H_cond_left we implement a DPCM
%encoder exploiting H_cond_up

R = [1:8];

xmin = 0;
xmax = 255;


for r = R

    %output
    Xout = NaN*ones(size(Xup));

    %first pixel: PCM
    step = (xmax-xmin)/2^r;
    Xout(1) = step*round(Xup(1)/step);

    %dpcm loop
    for ip = 2:M*N
        e = Xup(ip) - Xout(ip-1);
        eq = step*round(e/step);
        Xout(ip) = Xout(ip-1) + eq;
    end

    PSNR(r) = 10*log10(255^2/var(Xup-Xout));

end

plot(R,PSNR);
xlabel('Rate [bpp]');
ylabel('PSNR [dB]');
```